TP1 — Analyse d'un algorithme en fonctionnement

Matière : Les fondamentaux de l'IA **Niveau :** Bachelor 3 **Durée :** 30-45 min **Objectifs pédagogiques :** - Observer un pipeline ML complet de bout en bout - Mesurer des métriques de performance (accuracy, F1, confusion matrix) - Détecter le sur/sous-apprentissage (overfitting) - Initier l'explicabilité avec SHAP

Contexte

Ce TP ne demande **aucune expertise mathématique**. L'objectif est de **voir comment un algorithme apprend**, de mesurer ses performances avec des métriques adaptées, et de comprendre ses décisions grâce à l'explicabilité. Vous utiliserez Python, scikit-learn et SHAP comme boîtes noires intelligentes sur des données réelles.

Prérequis

```bash
pip install scikit-learn matplotlib pandas numpy seaborn shap
```

Ou directement sur **Google Colab** (aucune installation) : colab.research.google.com

Étape 1 — Charger les données : le dataset Iris

Le dataset **Iris** est un classique du ML : 150 mesures de fleurs (4 variables botaniques) à classer en 3 espèces. Chargé ici depuis le dépôt public UCI via seaborn-data.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix

# Chargement depuis URL publique — données réelles UCI (via seaborn-data)
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)

print("=== EXPLORATION DES DONNÉES ===")
print(f"Forme du dataset : {df.shape}")
print(f"\nDistribution des classes :")
print(df['species'].value_counts())
print(f"\nValeurs manquantes : {df.isnull().sum().sum()}")
print(f"\nAperçu :")
print(df.head(3))
print(f"\nStatistiques descriptives :")
print(df.describe())
```

Vérifiez et notez : - Combien d'échantillons par classe ? Le dataset est-il équilibré ? - Y a-t-il des valeurs manquantes ? - Quelles variables semblent a priori les plus discriminantes ?

Visualisation rapide des données

```python
# Pairplot : distributions et séparabilité des 3 espèces
sns.pairplot(df, hue='species', markers=["o", "s", "D"],
             plot_kws=dict(alpha=0.7), diag_kind='hist')
plt.suptitle("Dataset Iris — Séparabilité des classes", y=1.02)
plt.savefig('iris_pairplot.png', bbox_inches='tight')
plt.show()
```

Observez : Certaines paires de variables permettent-elles de séparer visuellement les 3 espèces ? Lesquelles semblent les plus efficaces ?

Étape 2 — Entraîner un modèle : baseline + modèle principal

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

le = LabelEncoder()
X = df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']].values
y = le.fit_transform(df['species'])
feature_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

# Split entraînement/test (80%/20%) stratifié
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Entraînement : {len(X_train)} échantillons | Test : {len(X_test)} échantillons")

# Normalisation (importante pour comparer les variables sur la même échelle)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

# Baseline simple : Decision Tree (max_depth=3)
dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train_sc, y_train)

# Modèle principal : Random Forest
rf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
rf.fit(X_train_sc, y_train)

print("Modèles entraînés.")
```

Question : Pourquoi commencer par une baseline simple (Decision Tree) avant un modèle plus complexe (Random Forest) ?

Étape 3 — Évaluer les métriques

```python
y_pred_dt = dt.predict(X_test_sc)
y_pred_rf = rf.predict(X_test_sc)

print("=== COMPARAISON BASELINE vs RANDOM FOREST ===")
for nom, pred in [("Decision Tree (baseline)", y_pred_dt), ("Random Forest       ", y_pred_rf)]:
    acc = accuracy_score(y_test, pred)
    f1  = f1_score(y_test, pred, average='weighted')
    print(f"{nom} → Accuracy : {acc*100:.1f}%  |  F1-score (weighted) : {f1:.3f}")

print("\n=== RAPPORT DÉTAILLÉ — RANDOM FOREST ===")
print(classification_report(y_test, y_pred_rf, target_names=le.classes_))

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Matrice de confusion — Random Forest')
plt.ylabel('Réalité')
plt.xlabel('Prédiction')
plt.tight_layout()
plt.savefig('confusion_matrix.png')
plt.show()
```

Questions : 1. Le Random Forest surpasse-t-il le Decision Tree ? Par quelle marge ? 2. Y a-t-il des espèces plus difficiles à classifier que d'autres ? Pourquoi selon vous ? 3. Quelle différence entre **accuracy** et **F1-score** ? Dans quel cas préférer l'un à l'autre ?

Étape 4 — Tuning des hyperparamètres et détection de l'overfitting

```python
from sklearn.model_selection import cross_val_score

# Impact du nombre d'arbres mesuré par validation croisée 5-fold
n_estimators_range = [10, 25, 50, 100, 200, 500]
scores_cv = []

for n in n_estimators_range:
    m  = RandomForestClassifier(n_estimators=n, random_state=42)
    cv = cross_val_score(m, X_train_sc, y_train, cv=5, scoring='accuracy')
    scores_cv.append(cv.mean())
    print(f"n_estimators={n:4d} → CV accuracy : {cv.mean()*100:.1f}% (±{cv.std()*100:.1f}%)")

# Courbe overfitting : score train vs score test selon la profondeur
profondeurs    = range(1, 20)
scores_train   = []
scores_test    = []

for d in profondeurs:
    m = RandomForestClassifier(n_estimators=50, max_depth=d, random_state=42)
    m.fit(X_train_sc, y_train)
    scores_train.append(m.score(X_train_sc, y_train))
    scores_test.append(m.score(X_test_sc,  y_test))

plt.figure(figsize=(9, 4))
plt.plot(profondeurs, scores_train, 'b-o', label='Score entraînement')
plt.plot(profondeurs, scores_test,  'r-o', label='Score test')
plt.xlabel('Profondeur maximale (max_depth)')
plt.ylabel('Accuracy')
plt.title('Underfitting vs Overfitting — Random Forest')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig('overfitting.png')
plt.show()
```

Questions : 1. À partir de quelle profondeur observe-t-on de l'overfitting (score train >> score test) ? 2. Quel nombre d'arbres offre le meilleur compromis performance / coût de calcul ? 3. Qu'apporte la **validation croisée** par rapport à un simple split train/test ?

Étape 5 — Explicabilité SHAP

```python
import shap

# Modèle final
rf_final = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
rf_final.fit(X_train_sc, y_train)

# Explainer SHAP adapté aux forêts aléatoires
explainer   = shap.TreeExplainer(rf_final)
shap_values = explainer.shap_values(X_test_sc)
# Pour Iris (3 classes), shap_values est une liste de 3 tableaux

# Summary plot global (importance par classe, vue en barres)
plt.figure()
shap.summary_plot(
    shap_values, X_test_sc,
    feature_names=feature_names,
    class_names=list(le.classes_),
    plot_type='bar',
    show=False
)
plt.title("SHAP — Importance globale des variables (3 classes)")
plt.tight_layout()
plt.savefig('shap_summary.png')
plt.show()

# Comparaison : feature importance sklearn (MDI) vs SHAP (classe virginica)
# Compatibilité SHAP < 0.42 (liste) et SHAP >= 0.42 (tableau 3D)
importances_sklearn = rf_final.feature_importances_
if isinstance(shap_values, list):
    importances_shap = np.abs(shap_values[2]).mean(axis=0)   # index 2 = virginica
else:
    importances_shap = np.abs(shap_values[:, :, 2]).mean(axis=0)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.barh(feature_names, importances_sklearn, color='steelblue')
ax1.set_title('Feature Importance — sklearn (MDI)')
ax1.set_xlabel('Importance')
ax2.barh(feature_names, importances_shap, color='darkorange')
ax2.set_title('Feature Importance — SHAP (classe virginica)')
ax2.set_xlabel('|SHAP value| moyen')
plt.tight_layout()
plt.savefig('feature_importance.png')
plt.show()
```

Questions : 1. Quelle variable est la plus déterminante selon SHAP ? Est-ce cohérent avec votre intuition ? 2. Quelle différence entre l'importance sklearn (MDI) et l'importance SHAP ? 3. Pourquoi l'explicabilité est-elle **cruciale** dans un contexte réglementé (médical, crédit bancaire) ?

Étape 6 — Debrief : 3 insights clés

Rédigez une synthèse structurée en **exactement 3 insights** issus de vos observations. Pour chacun, suivez ce format :

Insight 1 — Performance Observation : *(ce que vous avez mesuré)* Explication : *(pourquoi ce résultat)* Implication pratique : *(que faire concrètement ?)*

Insight 2 — Overfitting / Généralisation Observation : *(à partir de quelle profondeur ?)* Explication : *(qu'est-ce que l'overfitting ?)* Implication pratique : *(comment choisir la bonne complexité ?)*

Insight 3 — Explicabilité Observation : *(quelle variable domine selon SHAP ?)* Explication : *(pourquoi est-ce cohérent ou surprenant ?)* Implication pratique : *(dans quel contexte l'explicabilité est-elle obligatoire ?)*

Livrables

LIVRABLE	DESCRIPTION
Notebook propre	Code exécuté, cellules commentées, sans erreurs
Métriques détaillées	Tableau comparatif Decision Tree vs Random Forest (accuracy, F1)
Capture SHAP	**Screenshot du summary plot annoté**

Critères d'évaluation

CRITÈRE	POINTS
Code fonctionnel et exécutable	4
Analyse des métriques (accuracy, F1, confusion matrix)	4
Compréhension overfitting + graphique commenté	4
Explicabilité SHAP : capture + analyse	4
3 insights débrief : pertinence et justification	4
Total	**20**

Ressources

- scikit-learn — Documentation officielle
- SHAP — Documentation
- Google Colab
- Dataset Iris : UCI ML Repository via seaborn-data